Quora Insincere Questions Classification

A project work done in partial fulfilment of the "Certificate course on Data Analytics & Business Intelligence"

Submitted by:

Anjali (Roll No. DA8/22)

Certificate Course on Data Analytics & Business Intelligence

Shaheed Sukhdev College of Business Studies

July, 2023

Acknowledgement

I extend my sincere appreciation and gratitude to Dr. Rishi Rajan Sahay, who served as my mentor and guide throughout the Data Analytics course at Shaheed Sukhdev College of Business Studies. His unwavering support, insightful guidance, and vast knowledge greatly contributed to the successful completion of this project report.

I would also like to acknowledge the dedicated efforts of all the faculty members who taught various modules of the course. Their expertise and engaging teaching style provided me with a strong foundation in the field of data analytics.

I am thankful to the college for providing an enriching learning environment and valuable resources that played a pivotal role in enhancing my skills and knowledge.

Anjali

(DA8/22)

Date: 02/08/23

Declaration

I, Anjali, hereby declare that the project report titled " Quora Insincere Question Classification" submitted herewith is my original work completed as part of the on Data Analytics & Business Intelligence at Shaheed Sukhdev College of Business Studies. I have undertaken this project under the guidance and mentorship of Dr. Rishi Rajan Sahay, our esteemed instructor for the course.

The contents of this report reflect my understanding and interpretation of the concepts covered during the course. All references, sources, and contributions from external works have been duly cited and acknowledged. I affirm that this report has not been submitted previously for any other purpose, and I take full responsibility for the authenticity and accuracy of the information presented herein.

Anjali

(DA8/22)

Date: 02/08/23

Abstract

Quora is a widely-used platform for seeking and sharing knowledge through questions and answers. However, it faces challenges like identifying insincere or harmful content. Quora Insincere Classification employs machine learning to distinguish genuine questions from those with offensive or misleading intent. This safeguards user experience and content quality by automatically flagging inappropriate content, enhancing platform safety and credibility.

The project aimed to classify sincere and insincere questions on Quora using text data. After preprocessing, which involved tokenization, lowercasing, stemming, and lemmatization, the data was balanced by downsampling. Three classification models were evaluated: Logistic Regression using TF-IDF and Bag of Words, and XGBoost using Bag of Words. Hyperparameters were tuned using GridSearchCV for the Logistic Regression models and RandomizedSearchCV for XGBoost. ROC-AUC curves and heatmaps were visualized for model evaluation. The best-performing model, XGBoost, achieved a test AUC of around 0.96. Confusion matrices and classification reports were presented to assess model performance. This comprehensive project employed data preprocessing, feature engineering, model selection, hyperparameter tuning, and evaluation techniques to develop an effective classification solution for insincere question identification on the Quora platform.

Chapter 1

Aim and Introduction

1.1 Aim

The principle focus of our project is to perform data analysis over the insincere questions dataset provided by Quora and train two models using the most popular Machine Learning algorithm – Logistic Regression and XG Boost to predict the probability of question being insincere.

1.2 Introduction

Quora is a platform where people ask questions and get answers from other community members. It consists of a lot of domain experts which answer questions based on facts, their knowledge and experience. Questions are posted to acquire knowledge about the domain. But a lot of users spam the platform with insincere questions too. This not only degrades the integrity of the community but also results in financial loss to the platform as it loses the experts who get frustrated by these insincere questions and leave the platform. This affects its Search Engine Optimization rankings and the company loses revenue earned from the advertisements.

An insincere question is defined as a question intended to make a statement rather than look for helpful answers. Some characteristics that can signify that a question is insincere:

- Has a non-neutral tone
  - Has an exaggerated tone to underscore a point about a group of people
  - Is rhetorical and meant to imply a statement about a group of people
- Is disparaging or inflammatory
  - Suggests a discriminatory idea against a protected class of people, or seeks confirmation of a stereotype
  - Makes disparaging attacks/insults against a specific person or group of people
  - Based on an outlandish premise about a group of people
  - Disparages against a characteristic that is not fixable and not measurable
- Isn't grounded in reality
  - Based on false information, or contains absurd assumptions
- Uses sexual content (incest, bestiality, pedophilia) for shock value, and not to seek genuine answers

1.3 Dataset Description

This dataset is provided in the Kaggle competition "Quora Insincere Question Classification" and consists of approximately 1.3 million sample questions, each labeled as either insincere (positive example), or sincere (negative example), with approximately 94% of examples being sincere. Thus dataset is highly imbalanced.

Some example data points include:

Question	Class Name	Class Representation in Dataset
"What a difference between religion and spirituality?"	0	sincere
"Why are religious folks not considered clinically insane?"	1	insincere

1.4 Data fields

qid : unique question identifier

question_text : Quora question text

target : a question labeled "insincere" has a value of 1, otherwise 0

1.5 Data Reading

The dataset in csv format. It is read into the pandas Dataframe for easier processing and computations.

Chapter 2

Data Preprocessing

The dataset is in text format which means it cannot be directly read by the computer. So, it needs to be converted into a format so that computer can eaily interpret the semantic meaning of text. For that it needs to be preprocessed thoroughly so that while converting it to vectors, most of the data can be utilized. Thus, we follow a set of steps to preprocess data

2.1 Tokenization

Tokenization is the act of breaking up a sequence of strings into pieces such as words, keywords, phrases, symbols and other elements called tokens. Tokens can be individual words, phrases or even whole sentences. The tokens become the input for another process like parsing and text mining.

2.2 Lower Casing Tokens

The tokens will be converted into numerical vectors in later stages. For that the data needs to be uniform. The data cannot be vectorized properly if some tokens contains capital letters while some contain small. So, tokens are lower cased.

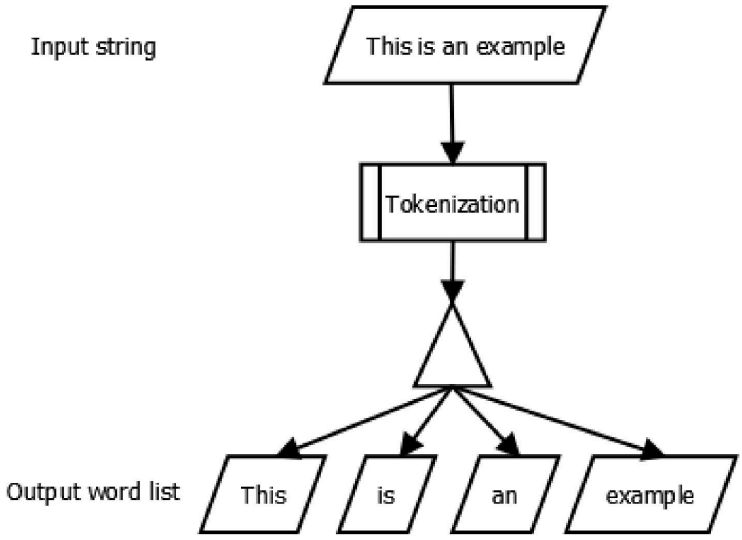

FIGURE 2.1: Tokenization

2.3 Removal of Non-Alphanumerics characters from Tokens

The tokens have been split up and lowercased. But they still contain punctuation marks which again affect vectorization. So, we remove punctuations too. So, only alphanumeric data remain in the tokens.

2.4 Remove Stopwords

Stopwords are the words in any language which does not add much meaning to a sentence. They can safely be ignored without sacrificing the meaning of the sentence. If we have a task of text classification or sentiment analysis then we should remove stop words as they do not provide any information to our model, i.e keeping out unwanted words out of our corpus, but if we have the task of language translation then stopwords are useful, as they have to be translated along with other words. Since we are solving text classification problem, we remove stopwords stored in NLTK package.

```
Output:
Tokenized text with stop words :
['Oh', 'man', ',', 'this', 'is', 'pretty', 'cool', '.', 'We', 'will', 'do', 'more', 'such', 'things', '.']
Tokenized text with out stop words :
['Oh', 'man', ',', 'pretty', 'cool', '.', 'We', 'things', '.']
```

FIGURE 2.2: Stopword Removal

2.5 Token Stemming

Stemming is the process of producing morphological variants of a root/base word. Stemming programs are commonly referred to as stemming algorithms or stemmers. A stemming algorithm reduces the words "chocolates", "chocolatey", "choco" to the root word, "chocolate" and "retrieval", "retrieved", "retrieves" reduce to the stem "retrieve". Stemming is an important part of the pipelining process in Natural language processing. We perform stemming using Porter Stemmer.

	original_word	stemmed_words
0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

FIGURE 2.3: Stemming

2.6 Token Lemmatization

Lemmatization, unlike Stemming, reduces the inflected words properly ensuring that the root word belongs to the language. In Lemmatization root word is called Lemma. A

lemma (plural lemmas or lemmata) is the canonical form, dictionary form, or citation form of a set of words. We perform lemmatization using WordNet Lemmatizer.

	original_word	lemmatized_word
0	goose	goose
1	geese	goose

FIGURE 2.4: Lemmatization

Chapter 3

Data Vectorization

Text Vectorization is the process of converting text into numerical representation. Machine learning algorithms operate on a numeric feature space, expecting input as a two-dimensional array where rows are instances and columns are features. In order to perform machine learning on text, we need to transform our documents into vector representations such that we can apply numeric machine learning. This process is called feature extraction or more simply, vectorization, and is an essential first step toward language-aware analysis.

Here is some popular methods to accomplish text vectorization:

3.1 Bag of Words

The Bag of Words (BoW) model is the simplest form of text representation in numbers. Like the term itself, we can represent a sentence as a bag of words vector (a string of numbers). It counts the words in the corpus and then replaces tokens with counts.

Let's recall the three types of movie reviews we saw earlier:

Review 1: This movie is very scary and long

Review 2: This movie is not scary and is slow

Review 3: This movie is spooky and good

Word	Count
This	3
movie	3
is	4
very	1
scary	2
and	3
long	1
not	1
slow	1
spooky	1
good	1

Vector of Review 1: [3 3 4 1 2 3 1]

Vector of Review 2: [3 3 4 1 2 3 4 1]

Vector of Review 3: [3 3 4 1 3 1]

3.2 Term Frequency - Inverse Document Frequency

TF-IDF is a statistical measure that evaluates how relevant a word is to a document in a collection of documents. This is done by multiplying two metrics: how many times a word appears in a document, and the inverse document frequency of the word across a set of documents.

It has many uses, most importantly in automated text analysis, and is very useful for scoring words in machine learning algorithms for Natural Language Processing (NLP).

TF-IDF (term frequency-inverse document frequency) was invented for document search and information retrieval. It works by increasing proportionally to the number of times a word appears in a document, but is offset by the number of documents that contain the word. So, words that are common in every document, such as this, what, and if, rank low even though they may appear many times, since they don't mean much to that document in particular.

TF-IDF for a word in a document is calculated by multiplying two different metrics:

The term frequency of a word in a document. There are several ways of calculating this frequency, with the simplest being a raw count of instances a word appears in a document. Then, there are ways to adjust the frequency, by length of a document, or by the raw frequency of the most frequent word in a document.

The inverse document frequency of the word across a set of documents. This means, how common or rare a word is in the entire document set. The closer it is to 0, the more common a word is. This metric can be calculated by taking the total number of documents, dividing it by the number of documents that contain a word, and calculating the logarithm. So, if the word is very common and appears in many documents, this number will approach 0. Otherwise, it will approach 1.

Multiplying these two numbers results in the TF-IDF score of a word in a document. The higher the score, the more relevant that word is in that particular document.

Chapter 4

Exploratory Data Analysis

4.1 Data Description

The dataset 1306122 data points out of which all **qid** values are unique and thus do not affect the sincerity of the question. So, they are dropped. The dataset does not have any null values or duplicate nows.

4.2 Class Distribution

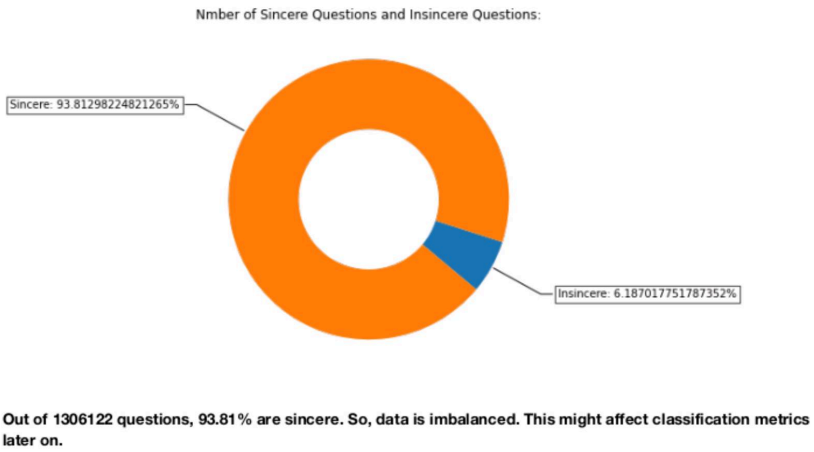

FIGURE 4.1: Class Distribution

4.3 Univariate Analysis

In univariate analysis, we will analyze the text data and its distribution after preprocessing.

4.3.1 Word Count Distribution of Clean Text

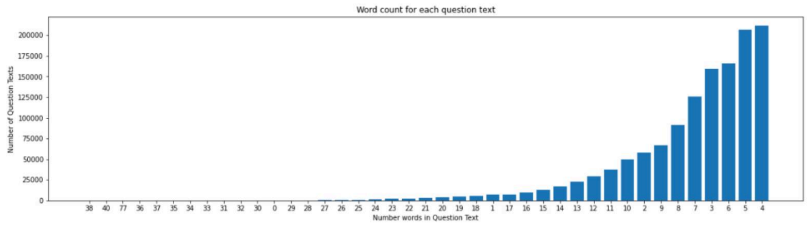

FIGURE 4.2: Word Count

Most of the clean text questions contain upto 15 words.

4.4 Bivariate Analysis

We analyze word distribution of both the classes together.

4.4.1 Probability Density Function - Clean Text Distribution

The insincere questions have wider spread towards right meaning that sincere questions are upto the point and insincere might have lot of non-sense words in them.

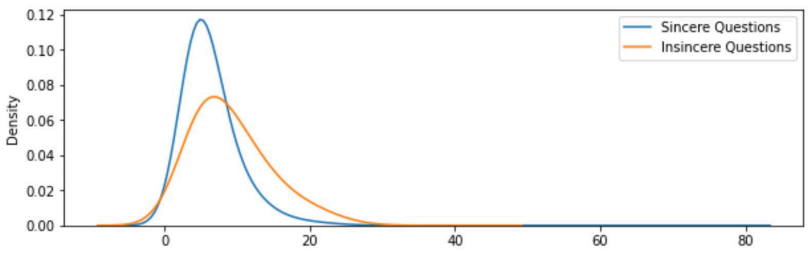

FIGURE 4.3: Probability Density Function

4.4.2 Cumulative Density Function - Clean Text Distribution

The CDF curve shows that at around 20 words, the probability of finding a sincere question is more compared to insincere question as they are supposed to have longer lengths as seen from previous PDF plot. We would see if the clean text with stopwords' distributions are similar to these plots or not.

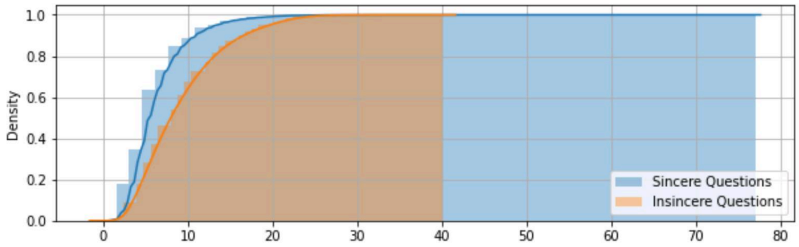

FIGURE 4.4: Cumulative Density Function

4.5 WordCloud

4.5.1 Insincere Questions

As seen in figure 3.5, the other top words in insincere questions are 'trump', 'women', 'white', 'quora', 'muslims' etc. This means the insincere questions would be mostly propaganda-based, politically influenced or racist.

4.5.2 Sincere Questions

As seen in figure 3.6, the other top words in sincere questions are 'best', 'good' etc. This means the sincere questions would be like "What is the best smartphone?"

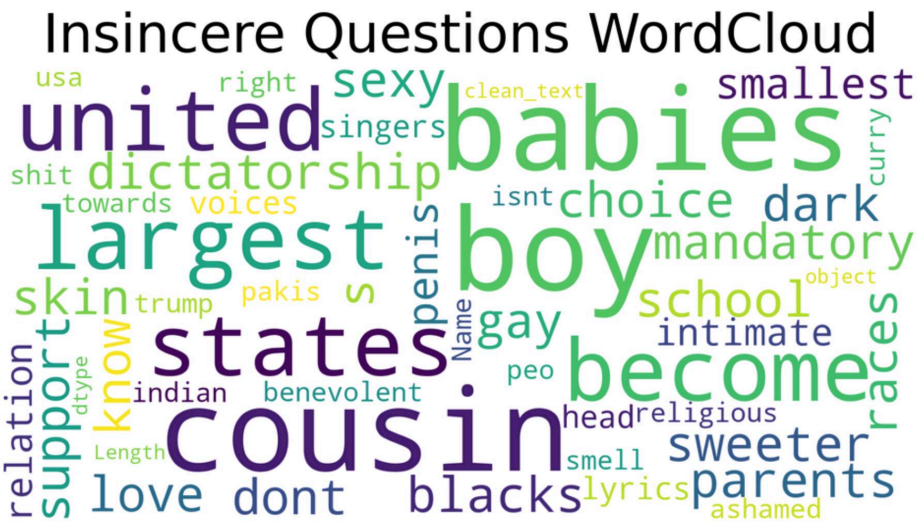

FIGURE 4.5: Insincere Questions WordCloud

FIGURE 4.6: Sincere Questions WordCloud

4.6 Boxplot Distribution - Clean Text

We can see that the insincere questions have more number of words as well as characters compared to sincere questions. So this would help while building model.

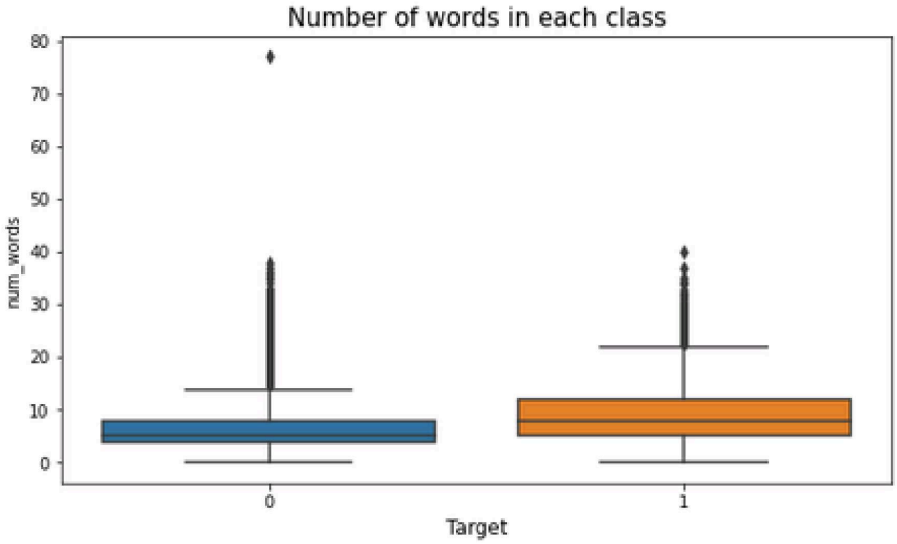

FIGURE 4.7: Sincere Questions WordCloud

4.7 t-Distributed Stochastic Neighbor Embedding

t-Distributed Stochastic Neighbor Embedding (t-SNE) is a non-linear technique for dimensionality reduction that is particularly well suited for the visualization of high-dimensional datasets. It is extensively applied in image processing, NLP, genomic data and speech processing.

We compute t-SNE over stemmed data. What it does is it reduces the dimensions from 5000 to 2 so that the data can be visualized clearly. This visualization does not give perfect boundaries but an estimate of how much separable the data is. TF-IDF embeddings for insincere questions are mostly bunched together such that a visible boundary can separate most of the data.

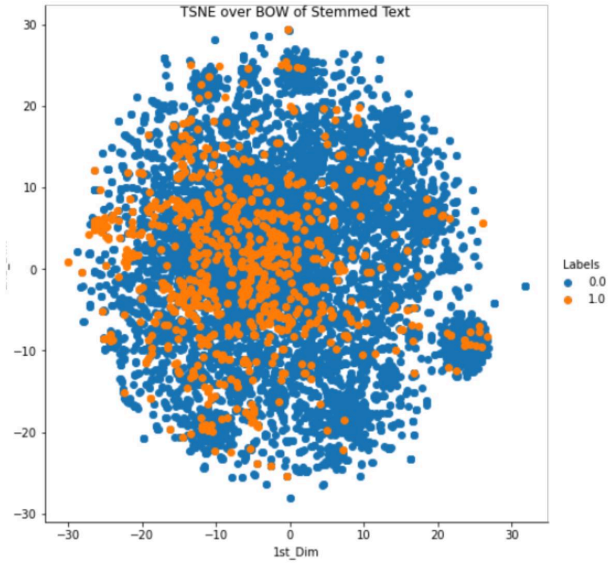

FIGURE 4.8: t-SNE over Bag of Words Encoding

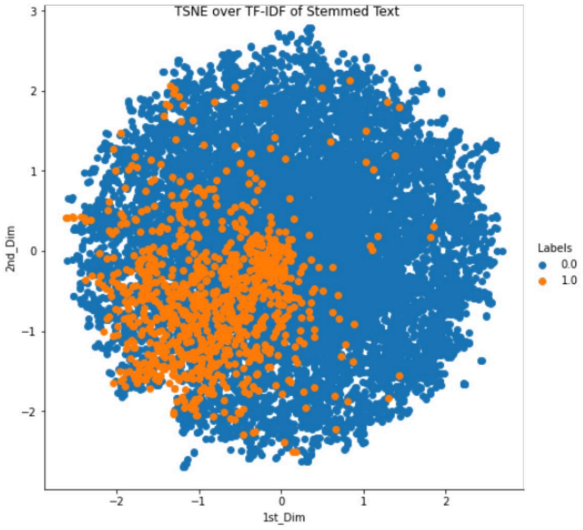

FIGURE 4.9: t-SNE over TF-IDF Encoding

Chapter 5

Building Models over Data

Since we have preprocessed data, analyzed it and vectorized it. Its time to build classification models over the data. These models will classify whether the question is insincere or not. We have 2 data vectors:

- Bag of Words over Stemmed Data
- TF-IDF over Stemmed Data

5.1 Data Downsampling

Since the data is extremely imbalanced (around 3%), we down sample data. The dataset if fairly large. So losing some data points while downsampling will not affect the performance much.

5.2 Train - Validation - Test Split

The dataset is further split into 3 parts in ratio 60:20:20 for training, test and validation. Training data is used to train the model, test data is used to check model performance

while training and validation data is used to find optimal hyperparameter, to prevent overfitting of model.

5.3 Hyperparameter Tuning

Parameters which define the model architecture are referred to as hyperparameters and thus this process of searching for the ideal model architecture is referred to as hyperparameter tuning. Hyperparameters are not model parameters and they cannot be directly trained from the data. Model parameters are inferenced during training when we optimize a loss function using something like gradient descent.

These hyperparameters might address model design questions such as:

- What degree of polynomial features should I use for my linear model?
- What should be the maximum depth allowed for my decision tree?
- What should be the minimum number of samples required at a leaf node in my decision tree?
- How many trees should I include in my random forest?
- How many neurons should I have in my neural network layer?
- How many layers should I have in my neural network?
- What should I set my learning rate to for gradient descent?

5.4 Logistic Regression

Logistic regression is a statistical model that in its basic form uses a logistic function to model a binary dependent variable, although many more complex extensions exist. In regression analysis, logistic regression[1] (or logit regression) is estimating the parameters

of a logistic model (a form of binary regression). Mathematically, a binary logistic model has a dependent variable with two possible values, such as pass/fail which is represented by an indicator variable, where the two values are labeled "0" and "1". In the logistic model, the log-odds (the logarithm of the odds) for the value labeled "1" is a linear combination of one or more independent variables ("predictors"); the independent variables can each be a binary variable (two classes, coded by an indicator variable) or a continuous variable (any real value). The corresponding probability of the value labeled "1" can vary between 0 (certainly the value "0") and 1 (certainly the value "1"), hence the labeling; the function that converts log-odds to probability is the logistic function, hence the name. The unit of measurement for the log-odds scale is called a logit, from logistic unit, hence the alternative names. Analogous models with a different sigmoid function instead of the logistic function can also be used, such as the probit model; the defining characteristic of the logistic model is that increasing one of the independent variables multiplicatively scales the odds of the given outcome at a constant rate, with each independent variable having its own parameter; for a binary dependent variable this generalizes the odds ratio.

5.4.1 Logistic Regression over Bag of Words - Stemmed Text

5.4.1.1 Hyperparameter Tuning and Training

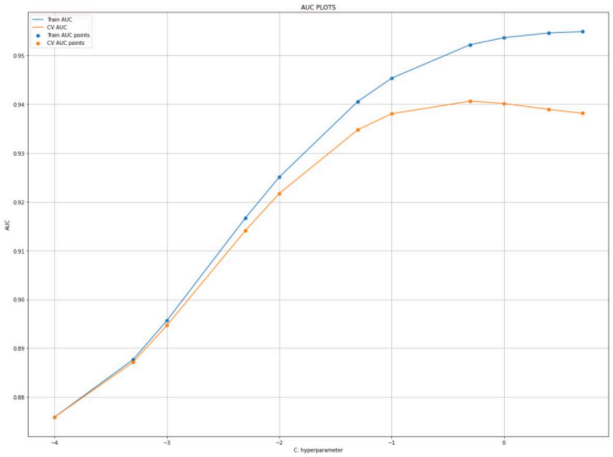

FIGURE 5.1: BoW - Stemmed Text: Hyperparameter Tuning

Best C is 0.5. We train Logistic Regression over the data.

5.4.1.2 Model Evaluation

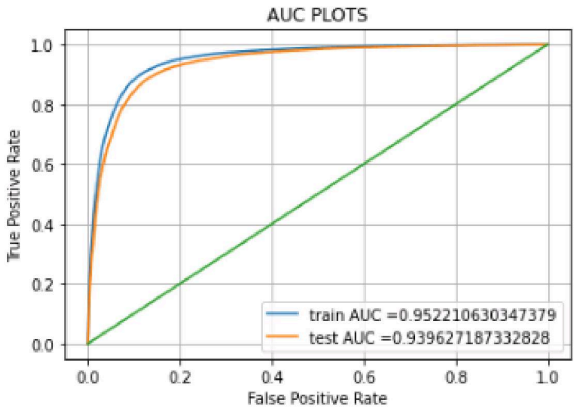

FIGURE 5.2: AUC Plots - Train and Test Data

5.4.1.3 Confusion Matrix

Confusion Matrix - Train Data

```
y_train_pred = lr.predict(x_train_bow)
```

```
confusion_matrix(y_train, y_train_pred)
```

```
array([[46898,  4944],
       [ 6419, 45175]])
```

Confusion Matrix - Test Data

```
y_test_pred = lr.predict(x_test_bow)
```

```
confusion_matrix(y_test, y_test_pred)
```

```
array([[14314,  1793],
       [ 2245, 13972]])
```

FIGURE 5.3: Confusion Matrix

5.4.1.4 Classification Report Matrix

Classification Report - Train Data

```
print(classification_report(y_train, y_train_pred))
```

	precision	recall	f1-score	support
0	0.88	0.90	0.89	51842
1	0.90	0.88	0.89	51594
accuracy			0.89	103436
macro avg	0.89	0.89	0.89	103436
weighted avg	0.89	0.89	0.89	103436

Classification Report - Test Data

```
print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.86	0.89	0.88	16107
1	0.89	0.86	0.87	16217
accuracy			0.88	32324
macro avg	0.88	0.88	0.88	32324
weighted avg	0.88	0.88	0.88	32324

FIGURE 5.4: Classification Report

5.4.2 Logistic Regression over TFIDF - Stemmed Text

5.4.2.1 Hyperparameter Tuning and Training

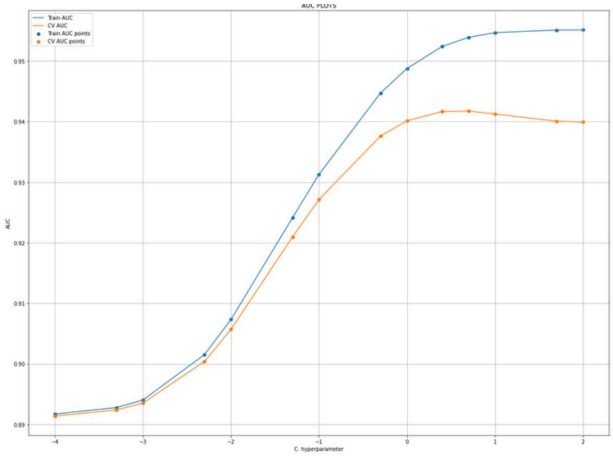

FIGURE 5.5: TFIDF - Stemmed Text: Hyperparameter Tuning

Best C is 5. We train Logistic Regression over the data.

5.4.2.2 Model Evaluation

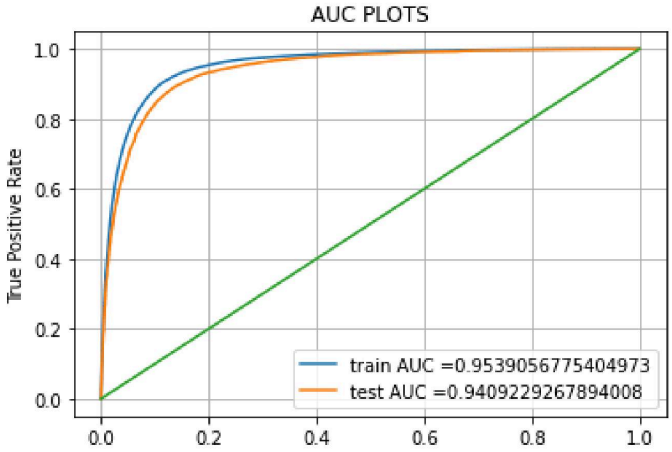

FIGURE 5.6: AUC Plots - Train and Test Data

5.4.2.3 Confusion Matrix

Classification Report - Train Data

```
print(classification_report(y_train, y_train_pred))
```

	precision	recall	f1-score	support
0	0.89	0.89	0.89	51842
1	0.89	0.89	0.89	51594
accuracy			0.89	103436
macro avg	0.89	0.89	0.89	103436
weighted avg	0.89	0.89	0.89	103436

Classification Report - Test Data

```
print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.88	0.88	0.88	16107
1	0.88	0.88	0.88	16217
accuracy			0.88	32324
macro avg	0.88	0.88	0.88	32324
weighted avg	0.88	0.88	0.88	32324

FIGURE 5.7: Confusion Matrix

5.4.2.4 Classification Report

Confusion Matrix - Train Data

```
y_train_pred = lr.predict(x_train_tfidf)
```

```
confusion_matrix(y_train, y_train_pred)
```

```
array([[46381,  5461],
       [ 5535, 46059]])
```

Confusion Matrix - Test Data

```
y_test_pred = lr.predict(x_test_tfidf)
```

```
confusion_matrix(y_test, y_test_pred)
```

```
array([[14107,  2000],
       [ 1983, 14234]])
```

FIGURE 5.8: Classification Report

5.5 XGBoost

5.5.1 XGBoost over Bag of Words - Stemmed Text

5.5.1.1 Hyperparameter Tuning and Training

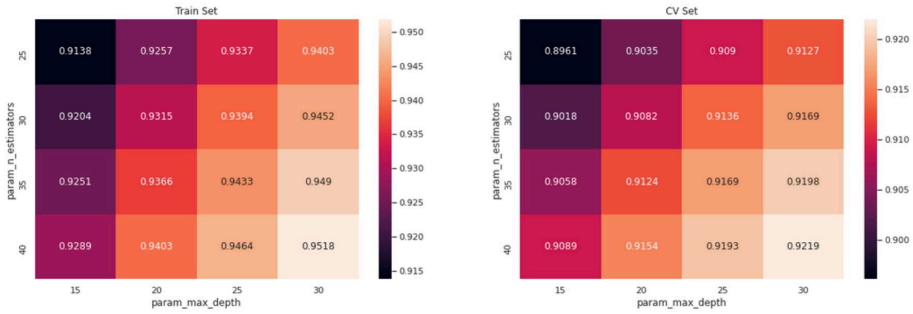

FIGURE 5.9: BoW - Stemmed Text: Hyperparameter Tuning

Maximum depth is 30 and number of estimators are 40. We train XGBoost over the data.

5.5.1.2 Model Evaluation

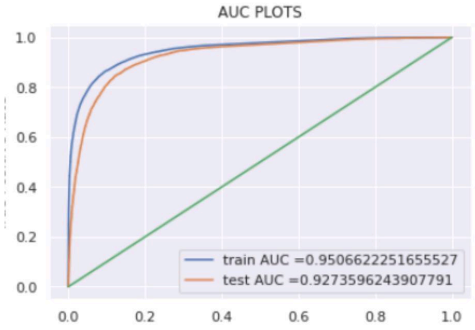

FIGURE 5.10: AUC Plots - Train and Test Data

5.5.1.3 Confusion Matrix

Confusion Matrix - Train Data

```
y_train_pred = classifier.predict(x_train_bow)
```

```
confusion_matrix(y_train, y_train_pred)
```

```
array([[59358,  5345],
       [ 9871, 54722]])
```

Confusion Matrix - Test Data

```
y_test_pred = classifier.predict(x_test_bow)
```

```
confusion_matrix(y_test, y_test_pred)
```

```
array([[14292,  1815],
       [ 2744, 13473]])
```

FIGURE 5.11: Confusion Matrix

5.5.1.4 Classification Report

Classification Report - Train Data

```
print(classification_report(y_train, y_train_pred))
```

	precision	recall	f1-score	support
0	0.86	0.92	0.89	64703
1	0.91	0.85	0.88	64593
accuracy			0.88	129296
macro avg	0.88	0.88	0.88	129296
weighted avg	0.88	0.88	0.88	129296

Classification Report - Test Data

```
print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.84	0.89	0.86	16107
1	0.88	0.83	0.86	16217
accuracy			0.86	32324
macro avg	0.86	0.86	0.86	32324
weighted avg	0.86	0.86	0.86	32324

FIGURE 5.12: Classification Report

Conclusion

In this extensive analysis, we explored various methods to classify insincere questions on the Quora platform. We began by preprocessing the data, including text tokenization, lowercasing, removing non-alphanumeric characters, stopping words, stemming, and lemmatization. Visualizations demonstrated the distribution of sincere and insincere questions and the word count distribution, offering insights into the dataset's characteristics.

We proceeded to build models using Logistic Regression and XGBoost on both TF-IDF and Bag of Words representations. Hyperparameter tuning was performed to enhance model performance. ROC curves, AUC scores, confusion matrices, and classification reports were employed to evaluate these models.

The XGBoost model on the Bag of Words representation yielded the highest AUC score of 88%, demonstrating its superior discriminatory power. These analyses provide valuable tools for identifying insincere content on Quora, which is crucial for maintaining the platform's integrity and user experience. By utilizing advanced techniques and models, Quora can effectively filter and moderate content to ensure a positive and trustworthy environment for its users.

References

1) Online Platforms:

Kaggle (data named ‘train’ taken from here), and other data science blogs often have articles related to NLP and machine learning techniques.

2) NLP and Machine Learning Books:

- "Natural Language Processing with Python" by Steven Bird, Ewan Klein, and Edward Loper.
- "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron.
- "Introduction to Machine Learning" by Alpaydin.